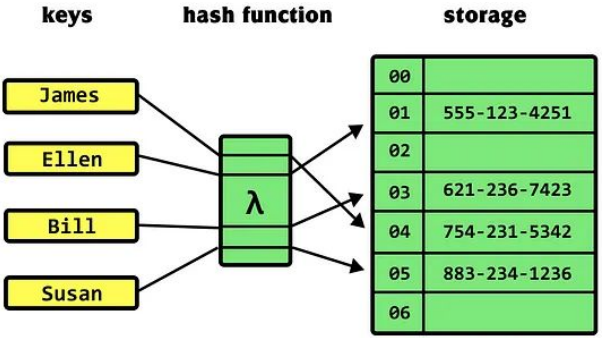
2.5.3 Составные типы. Мапы Карта. Словарь. Мапа

Мапа (или словарь) — это коллекция пар "ключ-значение".

Это одна из самых полезных структур данных в программировании, обеспечивающая быстрый доступ, добавление и удаление элементов.

До версии 1.24 в Go использовалась структура данных хеш-таблица.



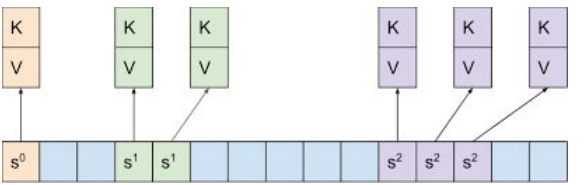
Как это работает простыми словами

1. **Хеширование:**
2. **Вставка в бакет**
3. **Увеличение мапы**

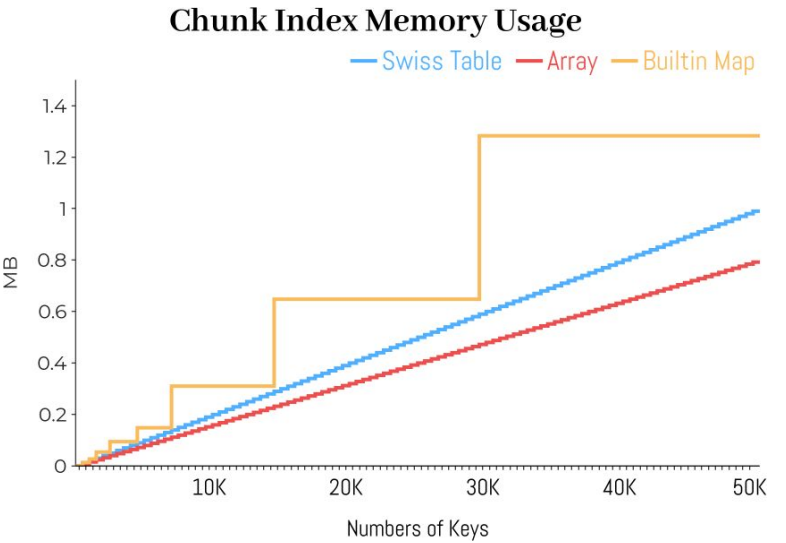


В версии Go 1.24 разработчики изменили способ работы карт (map), начав использовать **Swiss Table** — это новый способ организации хеш-таблиц.

Swiss Table использует плоский массив слотов, где каждый слот содержит одну пару ключ-значение и метаданные. Это изменение позволило сделать операции с картой быстрее и снизить задержки при росте.



Сравнение потребления памяти



MERION

Типы, которые можно использовать в качестве ключей

1. **Базовые типы**:

   - Целые числа: int, int8, int16, int32, int64, uint, uint8, uint16, uint32, uint64, uintptr. Числа с плавающей точкой: float32, float64.
   - Комплексные числа: complex64, complex128.
   - Строки: string.
   - Булевы значения: bool.

2. **Составные типы** (если их элементы сравнимы)

```
type Point struct {
    x int
    y int
}
m := make(map[Point]int)

m := make(map[[2]int]string) // Допустимо, так как [2]int — массив из сравнимых
элементов.


var i interface{} = 42
m := make(map[interface{}]int)
m[i] = 1 // Допустимо, так как 42 — сравнимый тип.
```

Типы, которые нельзя использовать в качестве ключей:

1. **Слайсы** ([]T):

```
m := make(map[[]int]string) // Ошибка компиляции: invalid map key type []int.
```

2. **Карты** (map[K]V):
3. **Функции**
4. **Структуры или массивы, содержащие несравнимые типы**:
5. **Типы с плавающей (NaN != NaN)**

Допустимые типы для значений (ValueType)

Для значений в map ограничений гораздо меньше. В качестве ValueType можно использовать **любой тип**, который допустим в Go, включая:

- Базовые типы: int, string, bool, float64 и т.д.
- Составные типы: слайсы, карты, структуры, указатели, функции, каналы, интерфейсы.
- Пользовательские типы.

Объявление мапы

```go
package main

import "fmt"

func main() {

    var m map[string]int
    m["Alice"] = 5
    fmt.Println(m)
}
```

Объявление и инициализация мапы

```go
package main

import "fmt"

func main() {

    var m map[string]int
    m = make(map[string]int)
    m["Alice"] = 5
    fmt.Println(m)
}
```

MERION

Инициализация и изменение элементов

```
package main

import "fmt"

func main() {

    marks := make(map[string]int)
    marks["Alice"] = 5
    marks["Bob"] = 3
    marks["Alice"] = 4
    fmt.Println(marks)
}
```

Удаление элемента

```
package main

import "fmt"

func main() {

    marks := make(map[string]int)
    marks["Alice"] = 5
    marks["Bob"] = 3
    delete(marks, "Bob")
    fmt.Println(marks)
}
```

Проверка наличия ключа

```
package main

func main() {

    marks := make(map[string]int)
    marks["Alice"] = 5
    marks["Bob"] = 3

    if value, exists := marks["Charlie"]; exists {
        println("Charlie mark:", value)
    } else {
        println("Charlie не найден.")
    }
}
```

Мапы передаются по ссылке

```
package main

import "fmt"

func modifyMap(m map[string]int) {
    m["apple"] = 100
}

func main() {
    m := map[string]int{"apple": 5, "banana": 3}
    modifyMap(m)
    fmt.Println(m)
}
```

Порядок элементов в мапе не гарантируется

```
package main

func main() {

    m := map[string]int{
        "apple":  5, "almond": 9,
        "banana": 3, "orange": 2,
        "grape":  7,
    }

     for key, value := range m {
        println(key, "->", value)
    }
}
```

Порядок элементов в мапе не гарантируется:

```
package main

func main() {
    m := map[int]bool{
        0: true,
        1: false,
        2: true,
    }
     for key, value := range m { if value {
            m[10+key] = true
        }
    }
}
```

```
map[0:true 1:false 2:true 10:true 12:true 20:true 22:true 30:true]
map[0:true 1:false 2:true 10:true 12:true 20:true 22:true 30:true 32:true]
map[0:true 1:false 2:true 10:true 12:true 20:true]
```

Неэффективная инициализация:

```
package main
import "fmt"

m := make(map[string]int)
for i := 0; i < 1000; i++ {
    key := fmt.Sprintf("key%d", i) m[key] = i
}

m := make(map[string]int, 1000)
for i := 0; i < 1000; i++ {
    key := fmt.Sprintf("key%d", i) m[key] = i
}
```